

User Manual

Introduction	3
System Architecture and Configuration	4
System Hardware Specifications	4
Service Nodes	4
Management Nodes	4
Storage Serever (OSS and MDS)	4
Login Nodes	4
Compute Nodes	5
Storage	5
Getting an access credential	6
Getting an Account on PRAJNA (AI/ML)	6
How to access the cluster	6
To access cluster from Windows OS:	6
PUTTY:	6
MobaXterm:	6
Command Prompt (Windows native application):	6
PowerShell (Windows native application)	7
To access cluster using Mac or Linux	7
Points to remember:	7
How to transfer files between local machine and HPC cluster	8
Resource Requests	9
SLURM	9
Commands	9
SLURM Partitions	10
Scheduling Type	10
Job Priority	10
Job Submission	11
Submitting Batch Scripts Jobs	11
1. Submitting a simple standalone job	11
2. Submit a job that's dependent on a prerequisite job being completed	11
3. Submitting multiple jobs:JOB ARRAY	12
Parameters used in SLURM job script	13
Sample SLURM Scripts for reference	14
Script for a Sequential Job	14
Script for a Parallel OpenMP Job	14
Script for Parallel Job – MPI (Message Passing Interface)	15
Script for Hybrid Parallel Job – (MPI + OpenMP)	15
Listing Partition	15
Monitoring jobs	15

Deleting jobs:	16
Holding a job:	16
Releasing a job:	17
Getting Node and Partition details	17
Migrate to SLURM	18
Using SPACK (package manager)	19
To Use Pre-Installed Applications from Spack	19
To load application	19
To list Pre-Loaded Application/Compilers	19
To install new application	19
Uninstalling Packages	20
Using Environments	20

Introduction

This document is the user manual for the **Prajna (AI-ML)** Supercomputing facility at IIT Bombay. It provides basic information required to utilize the supercomputer, such as information about logging on to the supercomputer, submitting jobs, retrieving the results on to the user's Laptop/ Desktop etc. In short, the manual describes steps to know to effectively utilize **Prajna (AI-ML)**.

The supercomputer **Prajna (AI-ML)** is based on a heterogeneous and hybrid configuration consisting of AMD and Intel processors, and NVIDIA A100, A40, L4, and L40S ADA GPU cards. It consists of multiple GPU-based nodes, including 9 DGX A100 nodes, 20 Exatron (A40) nodes, and 7 Exatron (L40S ADA) nodes.

System Architecture and Configuration

System Hardware Specifications

Prajna (AI-ML) system is based on a heterogeneous architecture comprising AMD and Intel processors, with a variety of NVIDIA GPUs (A100, A40, L4, L40S ADA). The cluster consists of compute nodes connected with a high-speed, low-latency InfiniBand interconnect network. The system uses the Lustre parallel file system.

- **DGX A100 Nodes: 9**
- **Exatron (A40) Nodes: 20**
- **Tyron (L4) Nodes: 10**
- **Exatron (L40S ADA) Nodes: 7**
- **Login Nodes: 2 Two Physical nodes**
- **Management Nodes: 2 Two Physical nodes**
- **Storage : 2 PetaByte Parallel File System**

Service Nodes

Management Nodes

In a cluster environment, management nodes act as the central control and coordination point for the entire cluster. They handle tasks like

- Providing login services into the computing system from the external network.
- Running the cluster administration and management tool kit.
- Booting compute nodes.
- Running the compiler stack, compiler and other license server
- Hosting the compilation of source code to native binary.
- Managing and maintaining the job submission scheduler
- Handling the distribution of the job.
- Creating and exporting file system on storage

Storage Serever (OSS and MDS)

A parallel file system is a type of file system that distributes data across multiple networked servers, allowing multiple clients to access and modify data concurrently. This architecture enables high-performance, scalable storage solutions, particularly for large-scale applications and clusters.

Files are divided into blocks and distributed across multiple storage nodes. Clients connect to the parallel file system through a network interface and can access files as if they were stored on a single server. Clients can perform read and write operations concurrently, utilizing the bandwidth of multiple storage nodes or object storage server (OSS). The system manages metadata, such as file locations and permissions, often using a metadata server (MDS) or distributed metadata.

Login Nodes

Login nodes are the nodes user gets to login and are typically used for administrative tasks such as editing, writing scripts, transferring files, monitoring/managing your jobs etc. You will always get connected to one of the login nodes. From the login nodes you submit a job and it is submitted to a compute/worker node and the compute/worker node runs your job. **For all users PRAJNA (AI/ML) Login Nodes are the entry points and hence are shared.** By default, there will be a limit on the resource and its utilization time that can be used. **If some user requests for a resource which is not available, his/her request are put in a queue.** Once a job is allocated resources it runs on the targeted worker/compute node(s).

Compute Nodes

GPU Compute Nodes feature accelerators cards that offer significant acceleration for parallel computing tasks using frameworks like CUDA and OpenCL. By harnessing the computational power of modern GPUs, these nodes are utilized for tasks such as scientific simulations, deep learning, and data analytics, providing high computational power and memory.

Storage

Parallel File System Storage Appliance with 2PB usable capacity with RAID6 (8+2) dual parity data protection. 400TB of NVMe flash storage with 1 Drive Writes Per Day endurance and 2PiB on 7.2K RPM 12 Gbps SAS disks and delivers sequential throughput of 60 GB/s (Gigabytes per second) with 100% sequential read and 60 GB/s (Gigabytes per second) with 100% sequential writes across the file system. It also supports metadata capacity to accommodate minimum 1 billion files on 1 DWPD NVMe flash with RAID6 (8+2) dual parity data protection.

Software Stack

Software Stack is an aggregation of software components that work together to accomplish various tasks. These tasks can range from facilitating users in executing their jobs to enabling system administrators to manage the system efficiently. Each software component within the stack is equipped with the necessary tools to achieve its specific task, and there may be multiple components of different flavors for different sub-tasks. Users have the flexibility to mix and match these components according to their preferences. For users, the primary focus is on preparing executables, executing them with their datasets, and visualizing the output. This typically involves compiling codes, linking them with communication libraries, math libraries, and numerical algorithm libraries, preparing executables, running them with desired datasets, monitoring job progress, collecting results, and visualizing output.

System administrators, on the other hand, are concerned with ensuring optimal resource utilization. To achieve this, they may require installation tools, health-check tools for all components, efficient schedulers, and tools for resource allocation and usage monitoring.

Figure 2- Software Stack

Functional Areas	Components
Base OS	Rocky 9
Architecture	X86_64
Provisioning and Cluster Manager	xCAT 2.16.5
Resource Manager	SLURM- 23.11.10

Getting an access credential

Getting an Account on PRAJNA (AI/ML)

You need to get an account on PRAJNA (AI/ML) to access the system. Follow the steps below to get an account to access the PRAJNA (AI/ML):

1. Visit hpcverse.iitb.ac.in
2. Go to "Quick Links" on the menu bar and navigate to "Account Request Form".
3. Click on the name PRAJNA.
4. Login using your LDAP credentials (Note that only faculty can login). This will direct one to the registration page of the server.
5. Fill in all required information on the registration page (note, DO NOT LEAVE ANY FIELD BLANK).
6. Once the form is complete, click on the "Submit" button.

How to access the cluster

To access cluster from Windows OS:

To access PRAJNA (AI/ML), there are few tools available, please see some below:

PUTTY:

1. Download PuTTY from its official website.
2. Install PuTTY on your computer.
3. Launch Putty from your desktop or Start menu.
4. In the dialog, locate the "Hostname or IP Address" input field.
5. Enter the hostname of the cluster or the IP address.
6. Select open, then enter your username and password when prompted
7. Press Enter to proceed with the connection.

MobaXterm:

1. Download MobaXterm from its official website.
2. Install MobaXterm on your computer.
3. Launch MobaXterm from your desktop or Start menu.
4. Click on the "Session" button in MobaXterm.
5. Enter the hostname/IP address, along with your username.
6. Enter the password when prompted.
7. Press Enter to proceed with the connection.

Command Prompt (Windows native application):

This is a native tool for Windows machines which can be used to login to the server:
Type the following command at the command prompt:

```
$> ssh username@serverIP
```

PowerShell (Windows native application)

This is a native tool for Windows machines which can be used to login to the server: Type the following command at the command prompt:

```
$> ssh username@serverIP
```

To access cluster using Mac or Linux

Both Mac and Linux systems provide a built-in SSH client. This eliminates the need to install any additional package. To connect to a SSH server, open the terminal and type the following command:

```
$> ssh username@serverIP
```

After getting credentials you may access the cluster, please remember the following points:

Points to remember:

- When you log in to the cluster, you will land on the login nodes. The login node serves as the primary gateway to the rest of the cluster, housing a job scheduler (known as SLURM) and other applications for creating and submitting the job. You can submit jobs to the queue, and they will execute when the required resources become available.
- Please refrain from running jobs directly on the login node. Login nodes are intended for compiling codes, transferring data and submitting jobs. If you run your job directly on the login node, it will be terminated.
- By default, two directories are available (i.e. /home and /scratch). These directories are available on the login node as well as the other nodes on the cluster.
 - /home is the directory which is primarily used for keeping your data.
 - /scratch is for temporary data storage, generally used to store interim data required for running jobs.
- Users are requested to regularly back up their own data in scratch directory free up space as jobs are finished. As per policy, any files not accessed in the last three months will be permanently deleted from /scratch.
- Whenever a newly created user on PRAJNA (AI/ML) attempts to log in with the user ID and temporary password provided via email by PRAJNA (AI/ML) support, it is mandatory for the user to change the password to one of their choosing. This ensures the security of your account. It is recommended to use a strong password containing a combination of lowercase and uppercase letters, numbers, and a few special characters that are easy for you to remember.
- Use the `passwd` command to change your user password. Enter your current password, followed by your new password, and then confirm the new password.
- Please open a ticket at <https://help.cc.iitb.ac.in/> or send a mail to hpc@iitb.ac.in regarding any concern you may have, and the support team will assist you with your problem.

How to transfer files between local machine and HPC cluster

Users need to have their data and applications related to their project or research work on PRAJNA (AI/ML). To store the data, special directories named “home” have been made available to the users. While these directories are common to all the users, each user will have their own directory with their groupname/username in the “/home/” directory, where they can store their data.

`/home/<groupname>/<username>/`

This directory is the place you will be placed when you login to the system. You can use this location for keeping your data, application etc.

However, there is a limit to the storage provided to users. The limits have been defined according to quota over these directories, and all groups will be allotted the same quota by default. When a user wishes to transfer data from their local system (laptop/desktop) to the HPC system, they can use various methods and tools.

A user using the ‘Windows’ operating system will have access to methods and tools native to Microsoft Windows, as well as tools that can be installed on their Windows machine.

Linux operating system users, however, do not require any tool. They can simply use the “scp” command on their terminal. Here’s how:

```
$> scp -r <path to the local data directory> <username>@10.195.100.101:
<path to directory on HPC where to save the data>
```


Resource Requests

A cluster is a group of computers that work together to solve complex computational tasks and presents itself to the user as a single system. For the resources of a cluster (e.g. CPUs, GPUs, memory) to be used efficiently, a resource manager (also called workload manager or batch-queuing system) is important. While there are many different resource managers available, the resource manager at PRAJNA (AI/ML) is SLURM.

SLURM

Slurm is an open source, fault-tolerant, and highly scalable cluster management and job scheduling system for large and small Linux clusters. As a cluster workload manager, Slurm has three key functions.

- First, it allocates exclusive and/or non-exclusive access to resources (compute nodes) to users for some duration of time so they can perform work.
- Second, it provides a framework for starting, executing, and monitoring work (normally a parallel job) on the set of allocated nodes.
- Finally, it arbitrates contention for resources by managing a queue of pending work.

Commands

sacct is used to report job or job step accounting information about active or completed jobs.

salloc is used to allocate resources for a job in real time. Typically this is used to allocate resources and spawn a shell. The shell is then used to execute *srun* commands to launch parallel tasks.

sattach is used to attach standard input, output, and error plus signal capabilities to a currently running job or job step. One can attach to and detach from jobs multiple times.

sbatch is used to submit a job script for later execution. The script will typically contain one or more *srun* commands to launch parallel tasks.

sbcast is used to transfer a file from local disk to local disk on the nodes allocated to a job. This can be used to effectively use diskless compute nodes or provide improved performance relative to a shared file system.

scancel is used to cancel a pending or running job or job step. It can also be used to send an arbitrary signal to all processes associated with a running job or job step.

scontrol is the administrative tool used to view and/or modify Slurm state. Note that many *scontrol* commands can only be executed as user root.

sinfo reports the state of partitions and nodes managed by Slurm. It has a wide variety of filtering, sorting, and formatting options.

sprio is used to display a detailed view of the components affecting a job's priority.

squeue reports the state of jobs or job steps. It has a wide variety of filtering, sorting, and formatting options. By default, it reports the running jobs in priority order and then the pending jobs in priority order.

srun is used to submit a job for execution or initiate job steps in real time. *srun* has a wide variety of options to specify resource requirements, including: minimum and maximum node count, processor count, specific nodes to use or not use, and specific node characteristics (so much memory, disk space, certain required features, etc.). A job can contain multiple job steps executing sequentially or in parallel on independent

or shared resources within the job's node allocation.

sshare displays detailed information about fairshare usage on the cluster. Note that this is only viable when using the priority/multifactor plugin.

sstat is used to get information about the resources utilized by a running job or job step.

strigger is used to set, get or view event triggers. Event triggers include things such as nodes going down or jobs approaching their time limit.

sview is a graphical user interface to get and update state information for jobs, partitions, and nodes managed by Slurm.

SLURM Partitions

Partition is a logical grouping of nodes that share similar characteristics or resources. Partitions are helpful to manage and allocate resources efficiently based on the specific requirements of jobs or users.

PRAJNA (AI/ML) consists of four types of computational nodes:

- DGX A100 – 9 nodes with 8×A100 GPU cards in each
- A40 – 20 nodes with 4×A40 GPU cards in each
- L4 – 10 nodes with 2×L4 GPU GPU cards in each
- L40S – 7 nodes with 8×L40S GPU cards in each

For an updated list of job limit, walltime etc. please visit the site <https://hpcverse.iitb.ac.in/queue-policy/prajna>

Scheduling Type

PRAJNA (AI/ML) has been configured with Slurm's backfill scheduling policy. It is good for ensuring higher system utilization; it will start lower priority jobs if doing so does not delay the expected start time of any higher priority jobs. Since the expected start time of pending jobs depends upon the expected completion time of running jobs, reasonably accurate time limits are important for backfill scheduling to work well.

Job Priority

The job's priority at any given time will be a weighted sum of all the factors that have been enabled in the slurm.conf file. Job priority can be expressed as:

```
Job_priority = site_factor + (PriorityWeightAge) * (age_factor) +  
(PriorityWeightAssoc) * (assoc_factor) + (PriorityWeightFairshare) *  
(fair-share_factor) + (PriorityWeightJobSize) * (job_size_factor) +  
(PriorityWeightPartition) * (priority_job_factor) + (PriorityWeightQOS) *  
(QOS_factor) + SUM(TRES_weight_cpu * TRES_factor_cpu, ...  
TRES_weight_<type> * TRES_factor_<type>, ...) - nice_factor
```

All of the factors in this formula are floating point numbers that range from 0.0 to 1.0. The weights are unsigned, 32-bit integers. The larger the number, the higher the job will be positioned in the queue, and the sooner the job will be scheduled. A job's priority, and hence its order in the queue, can vary over time. For example, the longer a job sits in the queue, the higher its priority will grow when the age weight is non-zero.

Age Factor: The age factor represents the length of time a job has been sitting in the queue and eligible to run.

Association Factor: Each association can be assigned an integer priority. The larger the number, the greater the job priority will be for jobs that request this association. This priority value is normalized to the highest priority of all the association to become

the association factor.

Job Size Factor: The job size factor correlates to the number of nodes or CPUs the job has requested.

Nice Factor: Users can adjust the priority of their own jobs by setting the nice value on their jobs. Like the system nice, positive values negatively impact a job's priority and negative values increase a job's priority. Only privileged users can specify a negative value.

Partition Factor: Each node partition can be assigned an integer priority. The larger the number, the greater the job priority will be for jobs that request to run in this partition.

Quality of Service (QOS) Factor: Each QOS can be assigned an integer priority. The larger the number, the greater the job priority will be for jobs that request this QOS.

Fair-share Factor: The fair-share component to a job's priority influences the order in which a user's queued jobs are scheduled to run based on the portion of the computing resources they have been allocated and the resources their jobs have already consumed.

Job Submission

We can submit jobs through a SLURM script. Creating a SLURM script is the optimal way to submit a job to the cluster.

Submitting Batch Scripts Jobs

Here is the example of sample slurm script:

```
#!/bin/bash

#SBATCH -N 1 // number of nodes
#SBATCH --ntasks-per-node=1 // number of cores per node
#SBATCH --error=job.%J.err // name of output file
#SBATCH --output=job.%J.out // name of error file
#SBATCH --time=01:00:00 // time required to execute the program
#SBATCH --partition= <queue name> // specifies queue name (standard is the
    default partition if you do not specify any partition job will be
    submitted using default partition)
#SBATCH --qos= <queue name> // specifies queue name (standard is the
    default partition if you do not specify any partition job will be submitted
    using default partition)
```

We can consider four cases of submitting a job here:

1. Submitting a simple standalone job

This is a simple submit script which is to be submitted

```
$> sbatch slurm-job.sh
Submitted batch job 106
```

2. Submit a job that's dependent on a prerequisite job being completed

Consider a requirement of pre-processing a job before proceeding to actual processing. Pre processing is generally done on a single core. In this scenario, the actual processing script is dependent on the outcome of the pre-processing script. Here is an example.

```
#!/bin/bash

#SBATCH -p standard
#SBATCH -J simple // -J option is used to name the job
sleep 60

Submit the job using:

$> sbatch simple.sh
Submitted batch job 149
```

Now we'll submit another job that's dependent on the previous job. There are many ways to specify the dependency conditions, but the "singleton" method is the simplest. The Slurm `-d singleton` argument tells Slurm not to dispatch this job until all previous jobs with the same name have completed.

```
$> sbatch -d singleton simple.sh //may be used for first pre-processing
on a core and then submitting
Submitted batch job 150

$> squeue
JOBID PARTITION NAME USER ST TIME NODES NODELIST(REASON)
150 standard simple user1 PD 0:00 1 (Dependency)
149 standard simple user1 R 0:17 1 rpcn001
```

Once the prerequisite job finishes the dependent job is dispatched.

```
$> squeue
JOBID PARTITION NAME USER ST TIME NODES NODELIST(REASON)
150 standard simple user1 R 0:31 1 rpcn001
```

3. Submitting multiple jobs:JOB ARRAY

Job arrays offer a mechanism for submitting and managing collections of similar jobs quickly and easily; job arrays with millions of tasks can be submitted in milliseconds (subject to configured size limits). All jobs must have the same initial options (e.g. size, time limit, etc.)

As for example

```
#!/bin/bash

#SBATCH -N 1
#SBATCH --ntasks-per-node=48
#SBATCH --error=job.%A_%a.err
#SBATCH --output=job.%A_%a.out
#SBATCH --time=01:00:00
#SBATCH --partition=standard
#SBATCH --qos=standard

source /lustre-flash/apps/spack/share/spack/setup-env.sh

spack load <modules to be loaded for your job>
cd <working directory> //change to your required directory
export OMP_NUM_THREADS=${SLURM_ARRAY_TASK_ID}

/home/guest/Rajneesh/Rajneesh/md_omp
```

```
$> sbatch --array=1-3 -N1 slurm_array.sh
Submitted batch job 151
```

A maximum number of simultaneously running tasks from the job array may be specified using a "%" separator. For example "--array=0-15%4" will limit the number of simultaneously running tasks from this job array to 4.

Parameters used in SLURM job script

The job flags are used with the SBATCH command. The syntax for the SLURM directive in a script is "#SBATCH <flag>". Some of the flags are used with the srun and salloc commands.

	Flag Syntax	Description
partition	--partition=<partition name>	Partition is a queue for the jobs.
time	--time=01:00:00	Time limit for the job.
nodes	--nodes=2	Number of compute nodes for the job.
cpus/cores	--ntasks-per-node=8	Corresponds to the number of cores on the compute node.
resource feature	--gres=gpu:2	Request use of GPUs on the gpu compute nodes
account	--account=<group-slurm account>	User may belong to multiple accounts. If only one account is allocated, it will be set as the default.
job name	--job-name="lammps"	Name of the job.
error file	--error=<filename_pattern>	Instruct Slurm to connect the batch script's standard error directly to the file name specified in the "filename pattern". By default both standard output and standard error are directed to the same file.
output file	--output=<filename_pattern>	Instruct Slurm to connect the batch script's standard output directly to the file name specified in the "filename pattern". By default both standard output and standard error are directed to the same file.
node list	-w, --nodelist	Request a specific list of hosts.
mail-type	--mail-type=	Notify users by email when certain event types occur. Valid type values are NONE, BEGIN, END, FAIL, REQUEUE, ALL, TIME_LIMIT, TIME_LIMIT_90 (reached 90 percent of time limit), TIME_LIMIT_80 (reached 80 percent of time limit), and TIME_LIMIT_50 (reached 50 percent of time limit), and ARRAY_TASKS (send emails for each array task). Multiple type values may be specified in a comma separated list
mail-user	--mail-user=<user email>	User to receive email notification of state changes as defined by --mail type.
Reservation	--reservation=<reservation>	Allocate resources for the job from the named reservation.
Validate script	--test-only	Validate the batch script and return an estimate of when a job would be scheduled to run given the current job queue and all the other arguments specifying the job

		requirements. No job is actually submitted.
exclusive access to nodes	--exclusive	Exclusive access to compute nodes. The job allocation cannot share nodes with other running jobs

Sample SLURM Scripts for reference

Script for a Sequential Job

```
#!/bin/bash

#SBATCH -N 1 // number of nodes
#SBATCH --ntasks-per-node=1 // number of cores per node
#SBATCH --error=job.%J.err // name of output file
#SBATCH --output=job.%J.out // name of error file
#SBATCH --time=01:00:00 // time required to execute the program #SBATCH #SBATCH
#SBATCH --partition=<partition name> // specifies queue name
#SBATCH --qos=<partition name> // specifies queue name

source /lustre-flash/apps/spack/share/spack/setup-env.sh // To start environment

// To load the package //
spack load intel-oneapi-compilers
cd <Path of the executable>
a.out (Name of the executable)
```

Script for a Parallel OpenMP Job

```
#!/bin/bash
#SBATCH -N 1 // Number of nodes
#SBATCH --ntasks-per-node=48 // Number of core per node
#SBATCH --error=job.%J.err // Name of output file
#SBATCH --output=job.%J.out // Name of error file
#SBATCH --time=01:00:00 // Time take to execute the program
#SBATCH --partition=cpu // specifies partition name
#SBATCH --qos=cpu // specifies queue name

source /lustre-flash/apps/spack/share/spack/setup-env.sh // To start environment

spack load intel-oneapi-compilers // To load the package
cd <path of the executable>
or
cd $SLURM_SUBMIT_DIR //To run job in the directory from where it is submitted
export OMP_NUM_THREADS=<threads required> //Depending upon your requirement you
can change the number of threads. If total number of threads per node is more
than 48, multiple threads will share core(s) and performance may degrade)
<Your executable> //Name of the executable
```

Script for Parallel Job – MPI (Message Passing Interface)

```
#!/bin/sh
#SBATCH -N 16 // Number of nodes
#SBATCH --ntasks-per-node=48 // Number of cores per node
#SBATCH --time=06:50:20 // Time required to execute the program
#SBATCH --job-name=lammps // Name of application
#SBATCH --error=job.%J.err_16_node_48 // Name of the output file
#SBATCH --output=job.%J.out_16_node_48 // Name of the error file
#SBATCH --partition=<queue name> // Partition or queue name
#SBATCH --qos=<partition name> // specifies queue name

source /lustre-flash/apps/spack/share/spack/setup-env.sh// To start environment

spack load intel-oneapi-compilers // To load the package
// Below are Intel MPI specific settings //
export I_MPI_FALLBACK=disable
export I_MPI_FABRICS=shm:dapl
export I_MPI_DEBUG=9 // Level of MPI verbosity
cd $SLURM_SUBMIT_DIR //change to required path where command needs to be
                        executed
or
cd <to the working directory> // Example Command to run the lammps in Parallel
//
time mpiexec.hydra -n $SLURM_NTASKS -genv OMP_NUM_THREADS 1 <path_to_executable>
-in in.lj
```

Script for Hybrid Parallel Job – (MPI + OpenMP)

```
#!/bin/sh
#SBATCH -N 16 // Number of nodes
#SBATCH --ntasks-per-node=48 // Number of cores for node
#SBATCH --time=06:50:20 // Time required to execute the program
#SBATCH --job-name=lammps // Name of application
#SBATCH --error=job.%J.err_16_node_48 // Name of the output file
#SBATCH --output=job.%J.out_16_node_48 // Name of the error file
#SBATCH --partition=standard // Partition or queue name
#SBATCH --qos=standard // specifies queue name

spack load intel-oneapi-compilers // To load the package //change to script
submission directory
cd $SLURM_SUBMIT_DIR
// Below are Intel MPI specific settings //
export I_MPI_FALLBACK=disable
export I_MPI_FABRICS=shm:dapl
export I_MPI_DEBUG=9 // Level of MPI verbosity
export OMP_NUM_THREADS=24 //Possibly then total no. of MPI ranks will be =
(total no. of cores, in this case 16 nodes x 48 cores/node) divided by (no. of
threads per MPI rank i.e. 24)
// Example Command to run the lammps in Parallel //
time mpiexec.hydra -n 32 lammps.exe -in in.lj
```

Listing Partition

sinfo displays information about nodes and partitions allowing users to view available nodes in the partition within the cluster.

```
$> sinfo
PARTITION AVAIL TIMELIMIT NODES  STATE NODELIST
a40       up      infinite  20  idle  cn20-a40,cn21-a40,cn22-a40,cn23-a40,cn24-a40,
cn25-a40,cn26-a40,cn27-a40,cn28-a40,cn29-a40,cn30-a40,cn31-a40,cn32-a40,cn33-a40
,cn34-a40,cn35-a40,cn36-a40,cn37-a40,cn38-a40,cn39-a40
l40*      up      infinite   7  idle  cn40-l40,cn41-l40,cn42-l40,cn43-l40,cn44-l40,
cn45-l40,cn46-l40
```

Monitoring jobs

Monitoring jobs on SLURM can be done using the command **squeue**. The command

squeue provides high-level information about jobs in the Slurm scheduling queue (state information, allocated resources, runtime, etc).

```
$> squeue
JOBID PARTITION NAME USER      ST TIME NODES NODELIST(REASON)
56      140      test Testuser PD 0:00  1      cn40-140
```

The command `scontrol` provides even more detailed information about jobs and job steps.

It will report more detailed information about nodes, partitions, jobs, job steps, and configuration.

```
$> scontrol show job 59
JobId=59 JobName=interactive
  UserId=Testuser(10014) GroupId=testgrp(5001) MCS_label=N/A      Priority=100003
  Nice=0      Account=test      QOS=normal      JobState=      PENDING
  Reason=Job's_QOS_not_permitted_to_use_this_partition_(140_allows_140_not_norm
al) Dependency=(null)
  Requeue=1 Restarts=0 BatchFlag=0 Reboot=0 ExitCode=0:0
  RunTime=00:00:00 TimeLimit=2-00:00:00 TimeMin=N/A
  SubmitTime=2025-07-09T12:24:44 EligibleTime=2025-07-09T12:24:44
  AccrueTime=2025-07-09T12:24:44
  StartTime=Unknown EndTime=Unknown Deadline=N/A
  SuspendTime=None      SecsPreSuspend=0      LastSchedEval=2025-07-09T13:06:19
  Scheduler=Backfill:*
  Partition=140 AllocNode:Sid=login1:371844
  ReqNodeList=(null) ExcNodeList=(null)
  NodeList=
  NumNodes=1-1 NumCPUs=1 NumTasks=1 CPUs/Task=1 ReqB:S:C:T=0:0:*:*
  ReqTRES=cpu=1,mem=128G,node=1,billing=1,gres/gpu=1
  AllocTRES=(null)
  Socks/Node=* NtasksPerN:B:S:C=0:0:*:* CoreSpec=*
  MinCPUsNode=1 MinMemoryNode=128G MinTmpDiskNode=0
  Features=(null) DelayBoot=00:00:00
  OverSubscribe=OK Contiguous=0 Licenses=(null) Network=(null)
  Command=/bin/bash
  WorkDir=/home/testgrp/Testuser
  TresPerNode=gres/gpu:1
```

scontrol update job <jobid>- set <new attribute value>

The above command change attributes of submitted job. Like time limit, nodelist, number of nodes, etc. For example:

```
$> scontrol update jobid=89 set TimeLimit=4-00:00:00
```

Deleting jobs:

Use the `scancel` command to delete active jobs. Users can cancel their own jobs only.

```
$> scancel 135
$> squeue --me
JOBID PARTITION NAME USER ST TIME NODES NODELIST(REASON)
```

Holding a job:

Use the `scontrol` command to hold the job.

```
$> scontrol hold <jobid>
```



```
$> squeue
JOBID PARTITION NAME USER ST TIME NODES NODELIST(REASON) 139 standard simple
user1 PD 0:00 1 (Dependency) 138 standard simple user1 R 0:16 1 rpcn001
$> scontrol hold 139
$> squeue
JOBID PARTITION NAME USER ST TIME NODES NODELIST(REASON) 139 standard simple
user1 PD 0:00 1 (JobHeldUser) 138 standard simple user1 R 0:32 1 rpcn001
```

Releasing a job:

```
$> scontrol release 139
$> squeue
JOBID PARTITION NAME USER ST TIME NODES NODELIST(REASON) 139 standard simple user1
PD 0:00 1 (Dependency) 138 standard simple user1 R 0:46 1 rpcn001
```

Getting Node and Partition details

scontrol show node <node name> - shows detailed information about compute nodes.

```
$> scontrol show node cn20-a40
NodeName=cn20-a40 Arch=x86_64 CoresPerSocket=16
CPUAlloc=0 CPUEfctv=64 CPUTot=64 CPULoad=0.00
AvailableFeatures=(null)
ActiveFeatures=(null)
Gres=gpu:4
NodeAddr=cn20-a40 NodeHostName=cn20-a40 Version=24.05.6
OS=Linux 4.18.0-553.el8_10.x86_64 #1 SMP Fri May 24 13:05:10 UTC 2024
RealMemory=515317 AllocMem=0 FreeMem=511456 Sockets=2 Boards=1
State=IDLE ThreadsPerCore=2 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A
Partitions=a40
BootTime=2025-07-07T16:15:34 SlurmdStartTime=2025-07-08T12:44:45
LastBusyTime=2025-07-10T07:58:57 ResumeAfterTime=None
CfgTRES=cpu=64,mem=515317M,billing=64,gres/gpu=4
AllocTRES=
CurrentWatts=0 AveWatts=0
```

scontrol show partition <partition name>- shows detailed information about a specific partition

```
$> scontrol show partition l40
PartitionName=l40
AllowGroups=ALL AllowAccounts=ALL AllowQos=l40
AllocNodes=ALL Default=YES QoS=N/A
DefaultTime=NONE DisableRootJobs=NO ExclusiveUser=NO ExclusiveTopo=NO
GraceTime=0 Hidden=NO MaxNodes=UNLIMITED MaxTime=UNLIMITED MinNodes=0 LLN=NO
MaxCPUsPerNode=UNLIMITED MaxCPUsPerSocket=UNLIMITED
Nodes=cn40-l40,cn41-l40,cn42-l40,cn43-l40,cn44-l40,cn45-l40,cn46-l40
PriorityJobFactor=1 PriorityTier=1 RootOnly=NO ReqResv=NO OverSubscribe=NO
OverTimeLimit=NONE PreemptMode=OFF
State=UP TotalCPUs=384 TotalNodes=7 SelectTypeParameters=NONE
JobDefaults=(null)
DefMemPerNode=UNLIMITED MaxMemPerNode=UNLIMITED
TRES=cpu=384,mem=3540617M,node=7,billing=384,gres/gpu=56
```

Migrate to SLURM

Environment Variables	PBS/Torque	SLURM
Job Id	\$PBS_JOBID	\$SLURM_JOBID
Submit Directory	\$PBS_JOBID	\$SLURM_SUBMIT_DIR
Node List	\$PBS_NODEFILE	\$SLURM_JOB_NODELIST
Job Specification	PBS/Torque	SLURM
Script directive	#PBS	#SBATCH
Job Name	-N [name]	--job-name=[name] OR -J [name]
Node Count	-1 nodes=[count]	--nodes=[min[-max]] OR -N [min[-max]]
CPU count	-1 ppn=[count]	---ntasks-per-node=[count]
CPUs Per Task		--cpus-per-task=[count]

Environment Variables	PBS/Torque	SLURM
Memory Size	-1 mem-[MB]	--mem=[MB] OR --mem-per-cpu=[MB]
Wall Clock Limit	-1 walltime=[hh:mm:ss]	--time=[min] OR --mem-per-cpu=[MB]
Node Properties	-1 nodes=4.ppn=8:[property]	--constraint=[list]
Standard Output File	-o [file_name]	--output=[file_name] OR -o [file_name]
Standard Error File	-e [file_name]	--error=[file_name] OR -e {file_name}
Combine stdout/stderr	-j oe (both to stdout)	(This is default if you do not specify --error)
Job Arrays	-t [array_spec]	--array=[array_spec] OR -a [array_spec]
Delay Job Start	-a [time]	--begin=[time]

Using SPACK (package manager)

In PRAJNA (AI/ML) we are using Spack. The purpose of Spack is to provide freedom to users for loading required applications or packages of specific versions with all its dependencies in the user environment. Users can find the list of all installed packages with their specific versions and dependencies. This also specifies which version of the application is available for a given session. All applications and libraries are made available through Spack. A User has to load the appropriate package from the available packages.

Spack automates the download-build-install process for software - including dependencies - and provides convenient management of versions and build configurations. It is designed to support multiple versions and configurations of software on a wide variety of platforms and environments. It is designed for large supercomputing centers, where many users and application teams share common installations of software on clusters with exotic architectures, using libraries that do not have a standard ABI. Spack is non-destructive: installing a new version does not break existing installations, so many configurations can coexist on the same system.

```
$> source /lustre-flash/apps/spack/share/spack/setup-env.sh
```

To Use Pre-Installed Applications from Spack

```
$> spack find
```

The spack find command is used to query installed packages on PRAJNA (AI/ML). Note that some packages appear identical with the default output.

The -l flag shows the hash of each package, and

The -f flag shows any non-empty compiler flags of those packages.

To load application

```
$> spack load <application name@version>
```

To list Pre-Loaded Application/Compilers

```
$> spack find --loaded
```

To install new application

First check the available compilers in Spack with below command:

spack compilers

Spack manages a list of available compilers on the system, detected automatically from the user's PATH variable. The Spack compilers command is an alias for the command Spack compiler list.

```
$> spack compilers
==> Available compilers
-- gcc almalinux8-x86_64 -----
gcc@8.5.0 gcc@14.2.0 gcc@13.3.0 gcc@12.4.0
-- nvhpc almalinux8-x86_64 -----
nvhpc@24.11 nvhpc@23.11
-- oneapi almalinux8-x86_64 -----
oneapi@2025.0.1 oneapi@2024.2.1
```

Spack list

The `spack list` command shows available packages.

The `spack list` command can also take a query string. Spack automatically adds wildcards to both ends of the string, or you can add your own wildcards.

```
$> spack list
```

spack install

Below is an example of installation of package using spack:

```
$> spack install gromacs@2020.5 +cuda~mpi+blas %intel ^intel-mkl
```

Above command will install gromacs version 2020.5 with blas and cuda support and without MPI support.

For blas there are multiple providers like OpenBLAS, Intel MKL, amdblis, and essl, ^intel-mkl will tell spack to use intel-mkl for blas routines.

Operators in Spack

- % to select compiler out of available compilers
- ^ to use variant of package
- @ to define the version number of packages to be installed.
- + to enable variant for package
- ~ to disable variant for package

Uninstalling Packages

Uninstall packages that we may not need.

```
$> spack uninstall zlib %gcc@6.5.0 (type: y)
```

Using Environments

Spack has an environment feature in which you can group installed software. You can install software with different versions and dependencies in each environment and can change software to use at once by changing environments. You can create a Spack environment by **spack env create** command. You can create multiple environments by specifying different environment names here.

```
$> spack env create -d ./my_spack_env myenv
```

To activate the created environment, type `spack env activate`. Adding `-p` option will display the current activated environment on your console. Then, install software you need to the activated environment.

```
[username@login1 ~]$ spack env activate -p myenv  
[myenv][username@login1 ~]$ spack install xxxxx
```

You can deactivate the environment by `spack env deactivate`. To switch to another environment, type `spack env activate` to activate it.

```
[myenv][username@login1 ~]$ spack env deactivate  
[username@login1 ~]$
```

Use `spack env list` to display the list of created Spack environments.

```
[username@login1 ~]$ spack env list  
==> 2 environments myenv another_env
```